`

Name: Sumit singh **Class:** MCADS11 **Roll No:** 1250259060

MongoDB Assignment

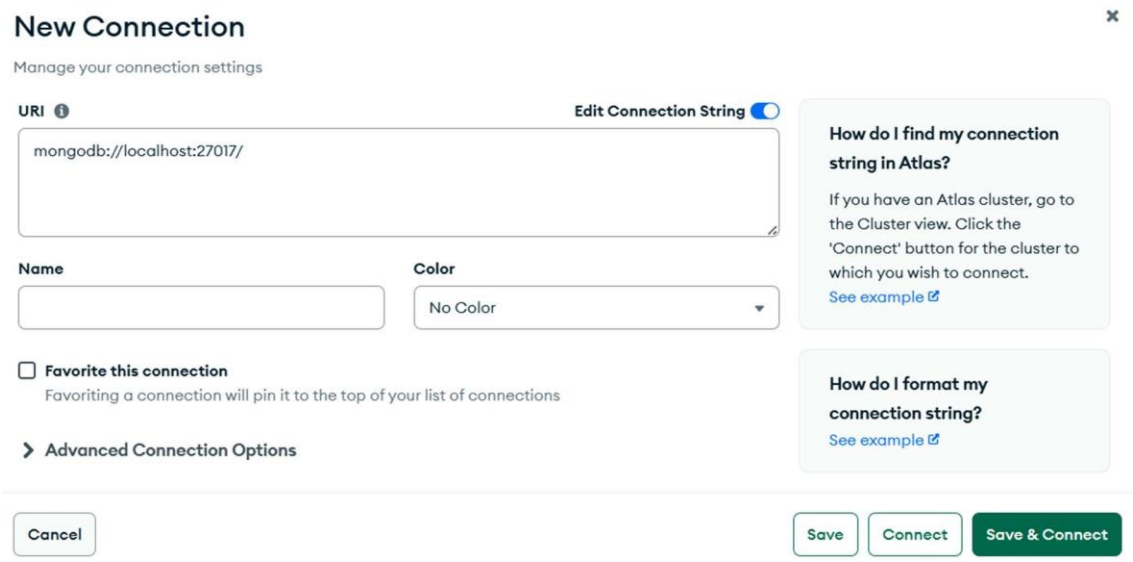
Data Collections (data sets) Assumed:

- **students**
- **faculty**
- **courses**
- **enrollments**
- **Activities**

Simple Steps to Create Database & Collection in MongoDB Compass

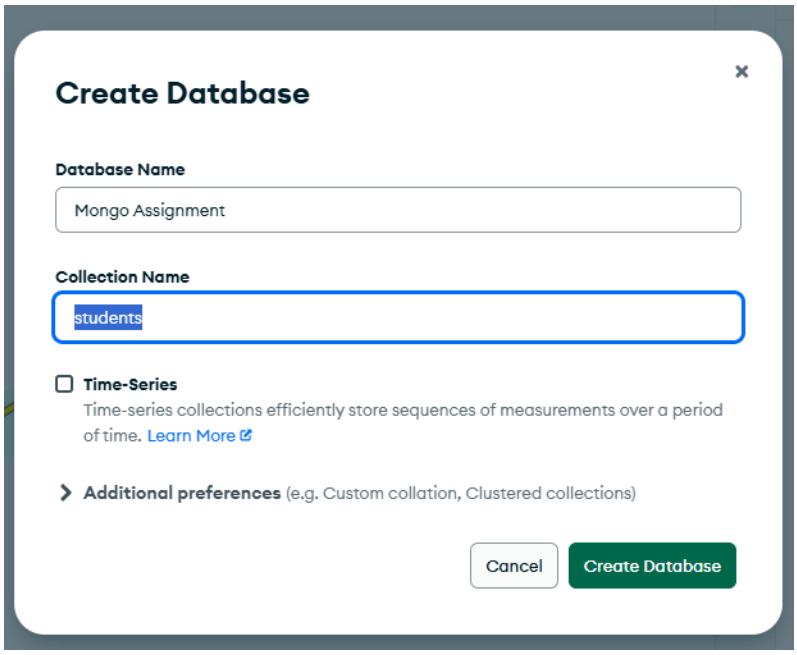
Step 1: Open & Connect

- Launch **MongoDB Compass**
- Enter connection string: mongodb://localhost:27017
- Click **Save & Connect**

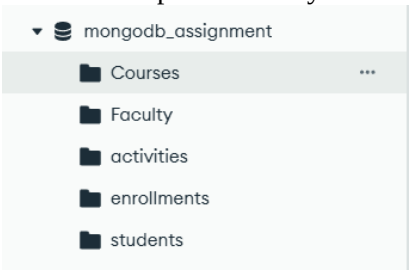


Step 2: Create Database

- In left sidebar, click + next to **"Databases"**
- **Database Name**: Mongo Assignment
- **Collection Name**: students
- Click **"Create Database"**

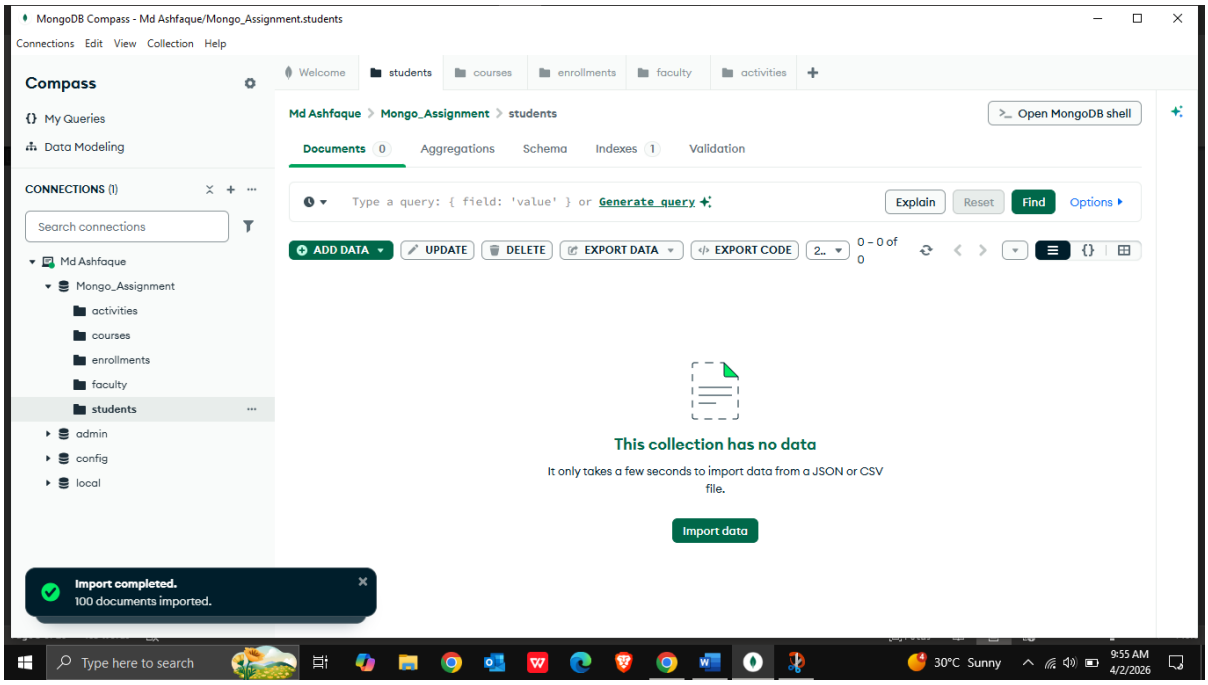


Step 3: Verify Creation

- New database appears in left sidebar
- Click it to expand → see your collection
- 

Step 4: Add Sample Document

- Click **"Add Data"** → **"Insert Document"**
- Add JSON


MongoDB Compass - Md Ashfaque/Mongo_Assignment.students
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
Md Ashfaque
Mongo_Assignment
activities
courses
enrollments
faculty
students
admin
config
local
Welcome
students
courses
enrollments
faculty
activities
Md Ashfaque > Mongo_Assignment > students
Open MongoDB shell
Documents 0
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT DATA
EXPORT CODE
0 - 0 of 0
This collection has no data
It only takes a few seconds to import data from a JSON or CSV file.
Import data
Import completed.
100 documents imported.
Type here to search
30°C Sunny
9:55 AM
4/2/2026

Write a Query for Each Question

1. Complex Filters & Projections

Question 1. List the names and departments of students who have more than 85% attendance and are skilled in both "MongoDB" and "Python".

```
>_MONGOSH
> use University
< switched to db University
> show collections
< activities
  course
  enrollment
  faculty
  student
> print("Name: Sumit Singh Roll no. 60")
< Name: Sumit Singh Roll no. 60
> db.students.find(
    {
      attendance: { $gte: 85 },
      skills: { $all: ["MongoDB", "Python"] }
    },
    {
      name: 1,
      department: 1,
      _id: 0
    }
  )
<
University>
```

No output means no students match both criteria in your data. Here's why and how to debug.

Question 2. Show all faculty who are teaching more than 2 courses. Display their names and the total number of courses they teach.

Input:

```
>_MONGOSH
> use mongodb_assignment
< switched to db mongodb_assignment
> db.Faculty.aggregate([
    {
      $project: {
        name: 1,
        totalCourses: { $size: "$courses" }
      }
    },
    {
      $match: {
        totalCourses: { $gt: 2 }
      }
    }
  ])
< {
```

Output:

```
{
  _id: 'F032',
  name: 'Julia Cole',
  totalCourses: 3
}
{
  _id: 'F040',
  name: 'Darrell Velasquez',
  totalCourses: 3
}
{
  _id: 'F048',
  name: 'Michael Poole',
  totalCourses: 3
}
{
  _id: 'F051',
  name: 'John Duran',
  totalCourses: 3
}
{
  _id: 'F061',
  name: 'Daniel Allen',
  totalCourses: 3
}
{
  _id: 'F083',
  name: 'Matthew Hanna',
  totalCourses: 3
```

2. Joins ($lookup) and Aggregations

Question 3. Write a query to show each student's name along with the course titles they are enrolled in (use $lookup between enrollments, students, and courses).

Input:

```
>_MONGOSH
> use mongodb_assignment
< switched to db mongodb_assignment
> db.students.aggregate([
  {
    $lookup: {
      from: "enrollments",
      let: { studentId: "$_id" },
      pipeline: [
        {
          $match: {
            $expr: { $eq: ["$student_id", "$$studentId"] }
          }
        }
      ],
      as: "enrollments"
    }
  },
  { $unwind: "$enrollments" },
  {
    $lookup: {
      from: "courses",
      let: { courseId: "$enrollments.course_id" },
      pipeline: [
        {
          $match: {
            $expr: { $eq: ["$_id", "$$courseId"] }
          }
        }
```

```
      ],
      as: "course"
    }
  },
  { $unwind: "$course" },
  {
    $project: {
      _id: 0,
      student_name: "$name",
      course_title: "$course.title"
    }
  }
])
```

Output:

```
< {
    student_name: 'Janet Jarvis',
    course_title: 'Configurable global framework'
  }
  {
    student_name: 'Janet Jarvis',
    course_title: 'Multi-lateral systemic success'
  }
  {
    student_name: 'Bruce Blair',
    course_title: 'Intuitive actuating superstructure'
  }
  {
    student_name: 'Bruce Blair',
    course_title: 'Balanced national function'
  }
  {
    student_name: 'Bruce Blair',
    course_title: 'Enhanced intangible emulation'
  }
  {
    student_name: 'Alexandra Bailey',
    course_title: 'Reactive neutral adapter'
  }
  {
    student_name: 'Tina Hodge',
    course_title: 'Streamlined zero administration strategy'
  }
```

Question 4. For each course, display the course title, number of students enrolled, and average marks (use $group).

Input

```
> db.enrollments.aggregate([
    {
      $group: {
        _id: "$course_id",
        totalStudents: { $sum: 1 },
        avgMarks: { $avg: "$marks" }
      }
    },
    {
      $lookup: {
        from: "courses",
        localField: "_id",
        foreignField: "_id",
        as: "courseDetails"
      }
    },
    {
      $unwind: "$courseDetails"
    },
    {
      $project: {
        _id: 0,
        courseTitle: "$courseDetails.title",
        totalStudents: 1,
        avgMarks: { $round: ["$avgMarks", 2] }
      }
    }
  ])
```

Output:

```
< {
    totalStudents: 2,
    courseTitle: 'Configurable scalable data-warehouse',
    avgMarks: 77.5
  }
  {
    totalStudents: 2,
    courseTitle: 'Profit-focused high-level capability',
    avgMarks: 58.5
  }
  {
    totalStudents: 2,
    courseTitle: 'Digitized even-keeled Internet solution',
    avgMarks: 75
  }
  {
    totalStudents: 1,
    courseTitle: 'Fully-configurable reciprocal installation',
    avgMarks: 53
  }
  {
    totalStudents: 1,
    courseTitle: 'De-engineered well-modulated installation',
    avgMarks: 75
  }
```

```
{
  totalStudents: 2,
  courseTitle: 'Customer-focused cohesive info-mediaries',
  avgMarks: 76.5
}
{
  totalStudents: 2,
  courseTitle: 'Integrated fault-tolerant task-force',
  avgMarks: 68.5
}
{
  totalStudents: 2,
  courseTitle: 'Streamlined scalable policy',
  avgMarks: 71.5
}
{
  totalStudents: 1,
  courseTitle: 'Enhanced radical secured line',
  avgMarks: 51
}
{
  totalStudents: 2,
  courseTitle: 'Optional next generation frame',
  avgMarks: 82
}
```

3. Grouping, Sorting, and Limiting

Question 5. Find the top 3 students with the highest average marks across all enrolled courses.

Input:

```
> db.enrollments.aggregate([
    {
      $group: {
        _id: "$student_id",
        avgMarks: { $avg: "$marks" }
      }
    },
    {
      $sort: { avgMarks: -1 }
    },
    {
      $limit: 3
    }
  ])
```

Output:

```
< {
    _id: 'S080',
    avgMarks: 100
  }
  {
    _id: 'S046',
    avgMarks: 98
  }
  {
    _id: 'S041',
    avgMarks: 94
  }
```

Question 6. Count how many students are in each department. Display the department with the highest number of students.

Input and Output:

```
db.students.aggregate([
  {
    $group: {
      _id: "$department",
      studentCount: { $sum: 1 }
    }
  },
  {
    $sort: { studentCount: -1 }
  },
  {
    $limit: 1
  }
])
{
  _id: 'Electrical',
  studentCount: 23
}
```

4. Update, Upsert, and Delete

Question 7. Update attendance to 100% for all students who won any "Hackathon".

Input and Output:

```
db.students.updateMany(
  {
    activities: {
      $elemMatch: { event: "Hackathon", position: "Winner" }
    }
  },
  { $set: { attendance: 100 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Question 8. Delete all student activity records where the activity year is before 2022.

Input and Output:

```
db.activities.deleteMany({
  activity_year: { $lt: 2022 }
})
{
  acknowledged: true,
  deletedCount: 0
}
```

Question 9. Upsert a course record for "Data Structures" with ID "C150" and credits 4—if it doesn't exist, insert it; otherwise update its title to "Advanced Data Structures".

Input and output:

```
db.courses.updateOne(
  { _id: "C150" },
  {
    $set: {
      title: "Advanced Data Structures",
      credits: 4
    },
    $setOnInsert: {
      _id: "C150"
    }
  },
  { upsert: true }
)
{
  acknowledged: true,
  insertedId: 'C150',
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

5. Array & Operator Usage

Question 10. Find all students who have "Python" as a skill but not "C++".

Input:

```
db.students.find(
  {
    skills: { $in: ["Python"], $nin: ["C++"] }
  },
  {
    name: 1,
    department: 1,
    skills: 1,
    _id: 0
  }
)
```

Output:

```
< {
    name: 'Kyle Hale',
    department: 'Electrical',
    skills: [
      'Python',
      'Java'
    ]
  }
  {
    name: 'Cody Whitehead',
    department: 'Biotechnology',
    skills: [
      'JavaScript',
      'Python'
    ]
  }
  {
    name: 'Thomas Jackson',
    department: 'Electrical',
    skills: [
      'Python',
      'AutoCAD'
    ]
  }
```

```
{
  name: 'Steven Wong',
  department: 'Biotechnology',
  skills: [
    'MongoDB',
    'Python'
  ]
}
{
  name: 'Cheryl Jackson',
  department: 'Civil',
  skills: [
    'Research',
    'Python'
  ]
}
{
  name: 'Mr. Darius Newman',
  department: 'Mechanical',
  skills: [
    'Python',
    'SQL'
  ]
}
```

Question 11. Return names of students who participated in "Seminar" and "Hackathon" both.

Input:

```
> db.activities.aggregate([
  {
    $match: {
      type: { $in: ["Seminar", "Hackathon"] }
    }
  },
  {
    $group: {
      _id: "$student_id",
      types: { $addToSet: "$type" }
    }
  },
  {
    $match: {
      types: { $all: ["Seminar", "Hackathon"] }
    }
  },
  {
    $lookup: {
      from: "students",
      localField: "_id",
      foreignField: "_id",
      as: "student"
    }
  },
  {
    $unwind: "$student"
  },
```

```
    {
      $project: {
        _id: 0,
        name: "$student.name"
      }
    }
  ])
```

Output:

```
< {
    name: 'Adam Solomon'
  }
  {
    name: 'Lydia Day'
  }
  {
    name: 'Taylor Webb'
  }
  {
    name: 'Patricia Scott'
  }
  {
    name: 'Carlos Bryant'
  }
```

6. Subdocuments and Nested Conditions

Question 12. Find students who scored more than 80 in "Web Development" only if they belong to the "Computer Science" department.

Input & Output:

```
> db.enrollments.aggregate([
    {
      $match: { marks: { $gt: 80 } }
    },
    {
      $lookup: {
        from: "courses",
        localField: "course_id",
        foreignField: "_id",
        as: "course"
      }
    },
    { $unwind: "$course" },

    // 🔴 FIX HERE (use actual title)
    {
      $match: { "course.title": "Enhanced real-time website" }
    },

    {
      $lookup: {
        from: "students",
        localField: "student_id",
        foreignField: "_id",
        as: "student"
      }
    },
    { $unwind: "$student" },
```

```
    {
      $match: { "student.department": "Computer Science" }
    },

    {
      $project: {
        _id: 0,
        name: "$student.name",
        department: "$student.department",
        marks: 1
      }
    }
  ])
<
> db.courses.find({title: "Web Development"}).count()
< 0
```

7. Advanced Aggregation (Challenge Level)

Question13. For each faculty member, list the names of all students enrolled in their courses along with average marks per student per faculty.

Input and

```
db.faculty.aggregate([
  // Join courses taught by faculty
  {
    $lookup: {
      from: "courses",
      localField: "_id",
      foreignField: "faculty_id",
      as: "courses"
    }
  },
  { $unwind: "$courses" },

  // Join enrollments for those courses
  {
    $lookup: {
      from: "enrollments",
      localField: "courses._id",
      foreignField: "course_id",
      as: "enrollments"
    }
  },
  { $unwind: "$enrollments" },
```

```
  {
    $lookup: {
      from: "students",
      localField: "enrollments.student_id",
      foreignField: "_id",
      as: "student"
    }
  },
  { $unwind: "$student" },

  // Group by faculty + student to calculate avg marks
  {
    $group: {
      _id: {
        faculty: "$name",
        student: "$student.name"
      },
      avgMarks: { $avg: "$enrollments.marks" }
    }
  },
```

```
  {
    $group: {
      _id: "$_id.faculty",
      students: {
        $push: {
          name: "$_id.student",
          avgMarks: "$avgMarks"
        }
      }
    }
  },

  // Final projection
  {
    $project: {
      _id: 0,
      faculty: "$_id",
      students: 1
    }
  }
]);
```

Output:

```
< {
    students: [
      {
        name: 'Janet Jarvis',
        avgMarks: 67
      }
    ],
    faculty: 'Laura Flores'
  }
  {
    students: [
      {
        name: 'Aaron Marshall',
        avgMarks: 58
      },
      {
        name: 'Larry Ramsey',
        avgMarks: 94
      },
      {
        name: 'Danielle Rich',
        avgMarks: 60
      }
    ],
    faculty: 'Daniel Allen'
  }
```

```
  {
    students: [
      {
        name: 'Isaac Rivers',
        avgMarks: 80
      },
      {
        name: 'Christina Gordon',
        avgMarks: 57
      },
      {
        name: 'Chelsey Davis',
        avgMarks: 86
      },
      {
        name: 'Erin Harris',
        avgMarks: 91
      },
      {
        name: 'Ronald Trevino',
        avgMarks: 100
      }
    ],
    faculty: 'Charles Newton'
  }
```

Question14. Show the most popular activity type (e.g., Hackathon, Seminar, etc.) by number of student participants.

Input and Output:

```
> db.activities.aggregate([
    {
      $project: {
        type: 1,
        participantCount: {
          $size: { $ifNull: ["$participants", []] }
        }
      }
    },
    {
      $group: {
        _id: "$type",
        totalParticipants: { $sum: "$participantCount" }
      }
    },
    {
      $sort: { totalParticipants: -1 }
    },
    {
      $limit: 1
    }
  ])
< {
    _id: 'Workshop',
    totalParticipants: 0
  }
```